

ECE 220 Computer Systems & Programming

Lecture 12 — Recursion

October 6, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

Announcements

- **Midterm 1 is graded** — mean 86, median 89, standard deviation 11.88
- Scores and the scanned exams are on **Gradescope**
- **Regrade requests:** on Gradescope, accepted through **Sunday, Oct 11, 11:59 pm**

The Searching Problem

A sorted array `a` of `n` items and a value `item`: return an index where it sits, or `-1` if it is not there.

The Searching Problem

A sorted array `a` of `n` items and a value `item`: return an index where it sits, or `-1` if it is not there.

Solution from last lecture

```
int binary_search(int array[], int n, int item) {  
    int low = 0, high = n - 1, mid;  
    while (low <= high) {  
        mid = (low + high) / 2;  
        if (array[mid] == item)  
            return mid;  
        if (item < array[mid])  
            high = mid - 1;    /* first half */  
        else  
            low = mid + 1;    /* second half */  
    }  
    return -1;  
}
```

Called from `main`:

```
int idx = binary_search(a, SIZE, 23);
```

ECE ILLINOIS 3 / 16

Recursive Implementation of Binary Search

```
int recursive_bin_search(int array[], int item,
                        int low, int high) {
    int mid;
    if (low > high)
        return -1;           /* empty region */
    mid = (low + high) / 2;
    if (array[mid] == item)
        return mid;          /* found it */
    if (item < array[mid])
        high = mid - 1;      /* first half */
    else
        low = mid + 1;        /* second half */
    return recursive_bin_search(array, item,
                                low, high);
}
```

Called from main:

```
idx = recursive_bin_search(a, 23, 0, SIZE - 1);
```

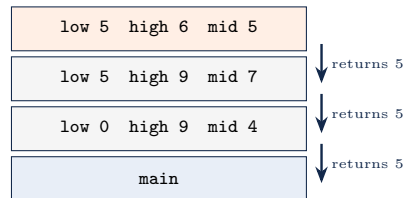
Recursive Implementation of Binary Search

```
int recursive_bin_search(int array[], int item,
                        int low, int high) {
    int mid;
    if (low > high)
        return -1;           /* empty region */
    mid = (low + high) / 2;
    if (array[mid] == item)
        return mid;          /* found it */
    if (item < array[mid])
        high = mid - 1;      /* first half */
    else
        low = mid + 1;       /* second half */
    return recursive_bin_search(array, item,
                                low, high);
}
```

Called from `main`:

```
idx = recursive_bin_search(a, 23, 0, SIZE - 1);
```

The run-time stack, searching for 23



Each call gets its own frame, with its own `low`, `high` and `mid` — which is why a function can call itself and not trip over its own variables.

The answer found at the top is handed back down, one frame at a time.

Generic Recursion

```
TYPE solve(args) {  
  if (base case)  
    return the answer directly;  
  if (another base case)  
    return the answer directly;  
  work before the calls, if any  
  r1 = solve(smaller args);  
  r2 = solve(other smaller args);  
  work after the calls, if any  
  return combine(r1, r2);  
}
```


Generic Recursion

```
TYPE solve(args) {  
    if (base case)  
        return the answer directly;  
    if (another base case)  
        return the answer directly;  
    work before the calls, if any  
    r1 = solve(smaller args);  
    r2 = solve(other smaller args);  
    work after the calls, if any  
    return combine(r1, r2);  
}
```

Three obligations

1. **One or more base cases**, each answered without calling
2. **One or more self-calls**, each on a smaller problem
3. Every path from a call reaches a base case

Miss any one of the three and the frames pile up until the runtime stack overflows.

Generic Recursion

```
TYPE solve(args) {  
    if (base case)  
        return the answer directly;  
    if (another base case)  
        return the answer directly;  
    work before the calls, if any  
    r1 = solve(smaller args);  
    r2 = solve(other smaller args);  
    work after the calls, if any  
    return combine(r1, r2);  
}
```

In `recursive_bin_search`: base cases _____, self-calls _____, work after the call _____.
Fibonacci, later today, has two of each; insertion sort has one base case, one call, and one insert afterwards.

Three obligations

1. **One or more base cases**, each answered without calling
2. **One or more self-calls**, each on a smaller problem
3. Every path from a call reaches a base case

Miss any one of the three and the frames pile up until the runtime stack overflows.

Today's Two Questions

**A recursive function is a function
that calls itself**

1. What makes a self-calling function *terminate* — and compute the right answer?
2. What actually happens on the run-time stack while it runs?

Recursion = Self-Call + Base Case

The same shape, on a number instead of an array:

The math:

```
RunningSum(1) = 1
RunningSum(n) =
    n + RunningSum(n-1)
```

- At least one **base case** that ends the process: here _____
- The recursive call must move *toward* it

The code:

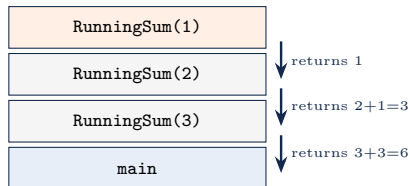
```
int RunningSum(int n) {
    /* base case: which n is easy? */
```

```
    /* otherwise: n plus what? */
```

```
}
```

What Happens on the Stack

RunningSum(3):



- Each call pushes a **fresh frame** — its own `n`, its own return address
- Deepest moment:

- The pending `n + ...` of every caller waits in its frame until the callee returns

No base case $\Rightarrow$ frames pile up until the runtime stack overflows — the error message is a literal description.

Fibonacci

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

$$F_n = F_{n-1} + F_{n-2}, \quad F_0 = 0, F_1 = 1$$

- How many base cases?

- Total calls to compute `Fib(5)`?

```
int Fibonacci(int n) {    /* n >= 0 */  
    /* base cases: which n need no recursion? */
```

```
    /* recursive case: the recurrence itself */
```

```
}
```

Fibonacci with a Lookup Table

```
int table[100];           /* main sets all to -1 */

int fibonacci(int n) {    /* 0 <= n < 100 */
    if (table[n] != -1)
        return table[n]; /* already known */
    if (n == 0)
        table[n] = 0;
    else if (n == 1)
        table[n] = 1;
    else
        table[n] = fibonacci(n - 1) + fibonacci(n - 2);
    return table[n];
}
```

- Remember every answer the first time you compute it (**memoization**)
- Calls for `fib(5)` now?

- Same recurrence, exponential → linear — bought with one global array

Insertion Sort, Nonrecursive

```
void insertion_sort(int array[], int n);
```

Last lecture's loop: u marks the first unsorted item, s scans the sorted part.

- When the outer loop reaches u , the first _____ items are already sorted
- Each pass of the outer loop does one _____
- The last pass, $u = n-1$, inserts $\text{array}[n-1]$ into _____

Sorting n items = sorting the first $n-1$, then one insert — a recurrence again, this time on a sort.

```
void insertion_sort(int array[], int n) {  
    int s, u, temp, empty_idx;  
    for (u = 1; u < n; u++) {  
        temp = array[u];      /* take next item */  
        empty_idx = u;  
        for (s = u - 1; s >= 0; s--)  
            if (temp < array[s]) {  
                array[s + 1] = array[s]; /* shift */  
                empty_idx = s;  
            }  
        array[empty_idx] = temp; /* insert */  
    }  
}
```


Insertion Sort, Step by Step

Blue is the sorted part, orange is the item just inserted.

5	2	6	1	3	9
---	---	---	---	---	---

the first item counts as sorted

Insertion Sort, Step by Step

Blue is the sorted part, orange is the item just inserted.

5	2	6	1	3	9
---	---	---	---	---	---

the first item counts as sorted

2	5	6	1	3	9
---	---	---	---	---	---

$u = 1$: take 2, shift 5 right

Insertion Sort, Step by Step

Blue is the sorted part, orange is the item just inserted.

5	2	6	1	3	9
---	---	---	---	---	---

the first item counts as sorted

2	5	6	1	3	9
---	---	---	---	---	---

$u = 1$: take 2, shift 5 right

2	5	6	1	3	9
---	---	---	---	---	---

$u = 2$: take 6, nothing to shift

Insertion Sort, Step by Step

Blue is the sorted part, orange is the item just inserted.

5	2	6	1	3	9
---	---	---	---	---	---

the first item counts as sorted

2	5	6	1	3	9
---	---	---	---	---	---

$u = 1$: take 2, shift 5 right

2	5	6	1	3	9
---	---	---	---	---	---

$u = 2$: take 6, nothing to shift

1	2	5	6	3	9
---	---	---	---	---	---

$u = 3$: take 1, shift 2 5 6 right

Insertion Sort, Step by Step

Blue is the sorted part, orange is the item just inserted.

5	2	6	1	3	9
---	---	---	---	---	---

the first item counts as sorted

2	5	6	1	3	9
---	---	---	---	---	---

$u = 1$: take 2, shift 5 right

2	5	6	1	3	9
---	---	---	---	---	---

$u = 2$: take 6, nothing to shift

1	2	5	6	3	9
---	---	---	---	---	---

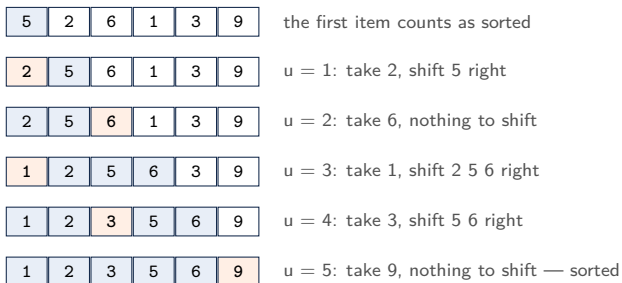
$u = 3$: take 1, shift 2 5 6 right

1	2	3	5	6	9
---	---	---	---	---	---

$u = 4$: take 3, shift 5 6 right

Insertion Sort, Step by Step

Blue is the sorted part, orange is the item just inserted.



Each pass inserts one item into the sorted part on its left; the outer loop walks u from 1 to $n-1$.

Insertion Sort, Recursive

```
void insertion_sort(int array[], int n);
```

Same contract, new body: sort the first $n - 1$ items *by calling yourself*, then insert the last one.

- **Base case:** $n \leq 1$ —

- **Recursive call:**

- **Insert** `array[n-1]`: the inner loop, unchanged, with $u =$ _____

```
void insertion_sort(int array[], int n) {  
    int s, temp, empty_idx;  
    /* base case: nothing left to sort? */
```

```
    /* recursive call: sort the first n-1 */
```

```
    /* insert array[n-1] into the sorted */  
    /* part, as the loop version did    */
```

```
}
```

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

`insertion_sort(a, 6)` sort the first 5 first

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

```
insertion_sort(a, 6)  sort the first 6 first
```

```
    insertion_sort(a, 5)
```

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

```
insertion_sort(a, 6)  sort the first 5 first
```

```
    insertion_sort(a, 5)
```

```
        insertion_sort(a, 4)
```

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

```
insertion_sort(a, 6)  sort the first 5 first
```

```
    insertion_sort(a, 5)
```

```
        insertion_sort(a, 4)
```

```
            insertion_sort(a, 3)
```

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

```
insertion_sort(a, 6)  sort the first 5 first
```

```
    insertion_sort(a, 5)
```

```
        insertion_sort(a, 4)
```

```
            insertion_sort(a, 3)
```

```
                insertion_sort(a, 2)
```

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

```
insertion_sort(a, 6)  sort the first 5 first
    insertion_sort(a, 5)
        insertion_sort(a, 4)
            insertion_sort(a, 3)
                insertion_sort(a, 2)
                    insertion_sort(a, 1)
```

the call with $n = 1$ has nothing to do: it returns at once

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

```
insertion_sort(a, 6) sort the first 5 first
    insertion_sort(a, 5)
        insertion_sort(a, 4)
            insertion_sort(a, 3)
                insertion_sort(a, 2)
                    insertion_sort(a, 1)
```

the call with $n = 1$ has nothing to do: it returns at once

Then each call, on its way back, inserts the one item it was holding.

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

2	5	6	1	3	9
---	---	---	---	---	---

$n = 2$ returns: inserted `a[1]`

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

```
insertion_sort(a, 6) sort the first 5 first
    insertion_sort(a, 5)
        insertion_sort(a, 4)
            insertion_sort(a, 3)
                insertion_sort(a, 2)
                    insertion_sort(a, 1)
```

the call with $n = 1$ has nothing to do: it returns at once

Then each call, on its way back, inserts the one item it was holding.

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

2	5	6	1	3	9
---	---	---	---	---	---

$n = 2$ returns: inserted $a[1]$

2	5	6	1	3	9
---	---	---	---	---	---

$n = 3$ returns: inserted $a[2]$

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

```
insertion_sort(a, 6) sort the first 5 first
```

```
    insertion_sort(a, 5)
```

```
        insertion_sort(a, 4)
```

```
            insertion_sort(a, 3)
```

```
                insertion_sort(a, 2)
```

```
                    insertion_sort(a, 1)
```

the call with $n = 1$ has nothing to do: it returns at once

Then each call, on its way back, inserts the one item it was holding.

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

2	5	6	1	3	9
---	---	---	---	---	---

$n = 2$ returns: inserted $a[1]$

2	5	6	1	3	9
---	---	---	---	---	---

$n = 3$ returns: inserted $a[2]$

1	2	5	6	3	9
---	---	---	---	---	---

$n = 4$ returns: inserted $a[3]$

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

```
insertion_sort(a, 6) sort the first 5 first
```

```
    insertion_sort(a, 5)
```

```
        insertion_sort(a, 4)
```

```
            insertion_sort(a, 3)
```

```
                insertion_sort(a, 2)
```

```
                    insertion_sort(a, 1)
```

the call with $n = 1$ has nothing to do: it returns at once

Then each call, on its way back, inserts the one item it was holding.

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

2	5	6	1	3	9
---	---	---	---	---	---

$n = 2$ returns: inserted $a[1]$

2	5	6	1	3	9
---	---	---	---	---	---

$n = 3$ returns: inserted $a[2]$

1	2	5	6	3	9
---	---	---	---	---	---

$n = 4$ returns: inserted $a[3]$

1	2	3	5	6	9
---	---	---	---	---	---

$n = 5$ returns: inserted $a[4]$

The Recursive Version, Step by Step

Calls go all the way down — no sorting happens on the way:

```
insertion_sort(a, 6) sort the first 5 first
    insertion_sort(a, 5)
        insertion_sort(a, 4)
            insertion_sort(a, 3)
                insertion_sort(a, 2)
                    insertion_sort(a, 1)
```

the call with $n = 1$ has nothing to do: it returns at once

Then each call, on its way back, inserts the one item it was holding.

The same six states as the loop version — the same work, described from the other end.

5	2	6	1	3	9
---	---	---	---	---	---

the array as it arrives

2	5	6	1	3	9
---	---	---	---	---	---

$n = 2$ returns: inserted $a[1]$

2	5	6	1	3	9
---	---	---	---	---	---

$n = 3$ returns: inserted $a[2]$

1	2	5	6	3	9
---	---	---	---	---	---

$n = 4$ returns: inserted $a[3]$

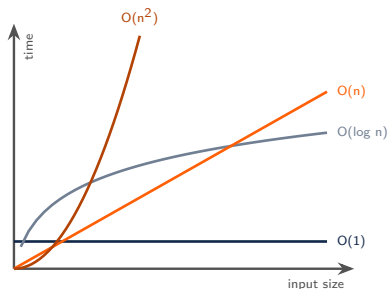
1	2	3	5	6	9
---	---	---	---	---	---

$n = 5$ returns: inserted $a[4]$

1	2	3	5	6	9
---	---	---	---	---	---

$n = 6$ returns: inserted $a[5]$

A Sneak Peek at Big-O



linear search

binary search

bubble / insertion sort

A vocabulary preview — ECE 374 makes it precise.
For now: the *shape* of the curve is what survives
when n gets big.

Summary & What's Next

Today

- **Binary search, twice:** as a loop, and as “probe the middle, then search the surviving half”
- **Recursion:** a base case, plus a self-call on a smaller problem — the code is the recurrence
- **On the stack:** every call gets a fresh frame; no base case means literal stack overflow
- **At work:** Fibonacci (and its exponential tree, tamed by a lookup table), insertion sort as a loop and as a recurrence
- **Big-O:** the growth curve is the algorithm's signature

Next lecture

- Recursion at full power: systematic trial and error — backtracking

Code from today: `bsearch.c`, `rbsearch.c`, `runsum.c`, `fib.c`, `fibtable.c`, `insertion.c`, `insertion_rec.c`.